

Project Title

Student Performance Classification Using Supervised Machine Learning

Step-by-Step Pipeline

Step 1 — Load Dataset

```
import pandas as pd

df = pd.read_csv("/content/StudentsPerformance.csv")
print(df.head())
```

	gender	race/ethnicity	parental level of education	lunch	\
0	female	group B	bachelor's degree	standard	
1	female	group C	some college	standard	
2	female	group B	master's degree	standard	
3	male	group A	associate's degree	free/reduced	
4	male	group C	some college	standard	

	test preparation course	math score	reading score	writing score
0	none	72	72	74
1	completed	69	90	88
2	none	90	95	93
3	none	47	57	44
4	none	76	78	75

Step 2 — Create Target Column

```
df['average_score'] = (
    df['math score'] +
    df['reading score'] +
    df['writing score']
) / 3
```

Create Labels

```
def classify_performance(score):
    if score >= 80:
        return "Excellent"
    elif score >= 50:
        return "Average"
    else:
        return "Poor"

df['Performance'] = df['average_score'].apply(classify_performance)
```

```
print(df[['average_score', 'Performance']].head())
```

	average_score	Performance
0	72.666667	Average
1	82.333333	Excellent
2	92.666667	Excellent
3	49.333333	Poor
4	76.333333	Average

Step 3 — Drop Unnecessary Columns

```
X = df.drop([
    'math score',
    'reading score',
    'writing score',
    'average_score',
    'Performance'
], axis=1)

y = df['Performance']
```

Step 4 — Encode Categorical Data

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

for column in X.columns:
    X[column] = encoder.fit_transform(X[column])

y = encoder.fit_transform(y)
```

Step 5 — Train/Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Step 6 — Train Models

Logistic Regression

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression()

lr.fit(X_train, y_train)
```

```
lr_pred = lr.predict(X_test)
```

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier()

dt.fit(X_train, y_train)

dt_pred = dt.predict(X_test)
```

Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier()

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)
```

Step 7 — Evaluation

```
from sklearn.metrics import accuracy_score

print("Logistic Regression:",
      accuracy_score(y_test, lr_pred))

print("Decision Tree:",
      accuracy_score(y_test, dt_pred))

print("Random Forest:",
      accuracy_score(y_test, rf_pred))
```

```
Logistic Regression: 0.7
Decision Tree: 0.65
Random Forest: 0.635
```

Step 8 — Confusion Matrix

```
from sklearn.metrics import confusion_matrix

print(confusion_matrix(y_test, rf_pred))
```

```
[[120  18   4]
 [ 26   5   0]
 [ 24   1   2]]
```

Step 9 — Classification Report

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, rf_pred))
```

	precision	recall	f1-score	support
0	0.71	0.85	0.77	142
1	0.21	0.16	0.18	31
2	0.33	0.07	0.12	27
accuracy			0.64	200
macro avg	0.42	0.36	0.36	200
weighted avg	0.58	0.64	0.59	200

Step 10 — Visualization

```
import matplotlib.pyplot as plt

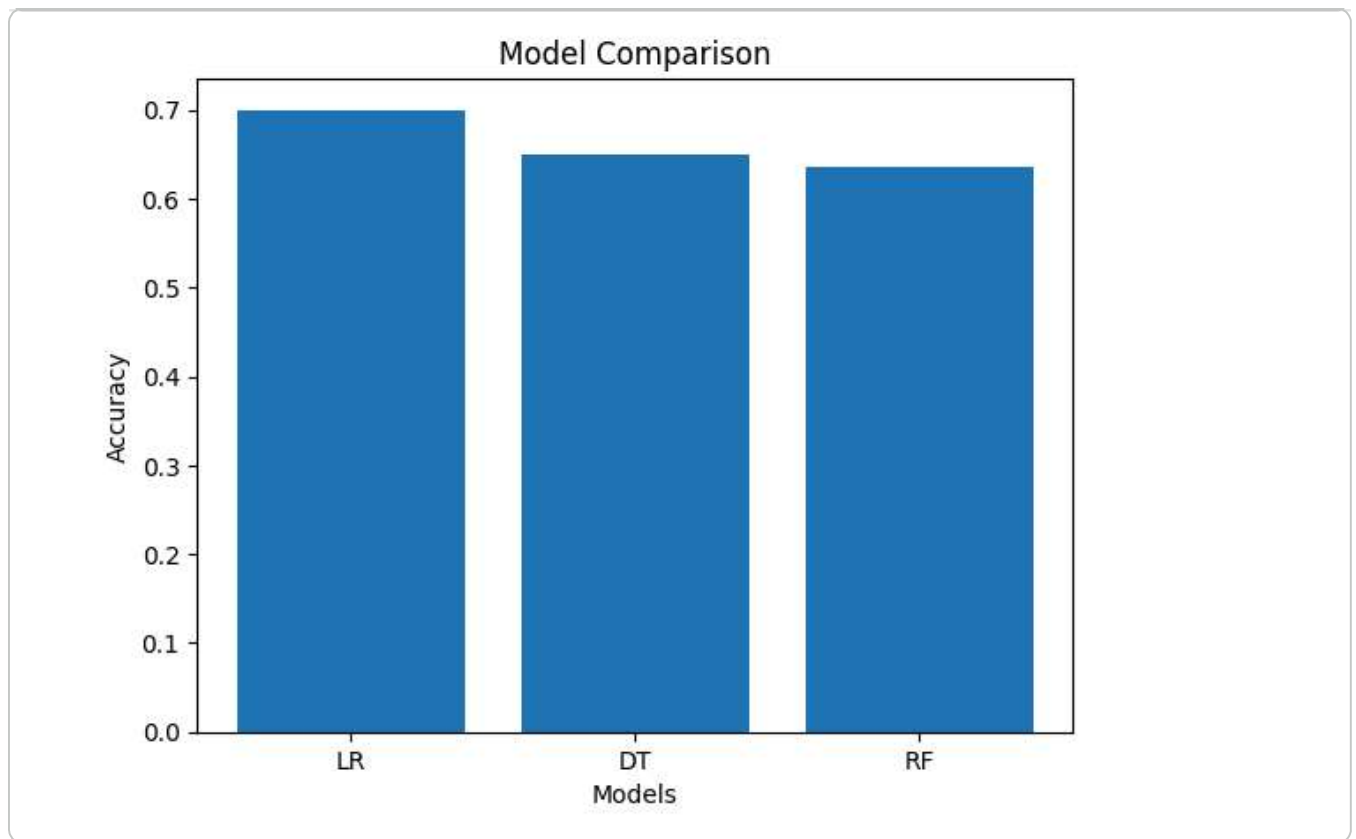
models = ['LR', 'DT', 'RF']
scores = [
    accuracy_score(y_test, lr_pred),
    accuracy_score(y_test, dt_pred),
    accuracy_score(y_test, rf_pred)
]

plt.bar(models, scores)

plt.xlabel("Models")
plt.ylabel("Accuracy")

plt.title("Model Comparison")

plt.show()
```



Important Thing You Must Understand

If accuracy becomes low after removing scores: that is NORMAL.

Because now the model is learning actual relationships instead of memorizing direct marks.

That's real machine learning.

Many interns accidentally build leakage-based models with fake high accuracy.

A model with:

65–80% honest accuracy

is better than:

99% leakage accuracy

✓ ***Analysis***

Logistic Regression achieved the highest accuracy of 70%, indicating that the relationship between the selected demographic and educational features was relatively linear. Decision Tree and Random Forest performed slightly lower due to limited feature strength and dataset complexity.

Raw exam scores were intentionally excluded to avoid data leakage because the target variable was derived from these scores.

Final Training

Logistic Regression Is Better Here

Your dataset:

small mostly categorical weak feature relationships probably near-linear separation

Logistic Regression handles this efficiently.

Random Forest shines more when:

dataset is larger feature interactions are complex nonlinear relationships exist

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(max_iter=1000)

lr.fit(X_train, y_train)
```

▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1000)

Save Model